

Projektbeschreibung – Entwicklung eines Webshops

Inhaltsverzeichnis

1. Einleitung	1
2. Allgemeine Konzepte, Funktionalitäten und Rollen	1
2.1. Allgemeine Konzepte	1
2.2. Rollen und deren Funktionalitäten	3
3. Architektur und Komponentenstruktur	3
3.1. Schnittstellen zwischen Backend und Frontend	3
3.2. Entwurfsmuster	3
4. Weitere Anforderungen	4
4.1. Allgemeine Anforderungen	4
4.2. Technische Anforderungen	4
5. Organisatorisches und Entwicklungsrichtlinien	5
5.1. Weitere wichtige Hinweise	5
5.2. Umgang mit Large Language Models (LLMs)	6
5.3. Erwartete Deliverables	6
5.4. Fristen, wichtige Termine und Abgaben	7
6. Hilfreiche Referenzen und Tools	8

1. Einleitung

Ziel dieses Projekts ist die Entwicklung einer Webapplikation für einen kleinen Online-Shop. Die Applikation soll die grundlegenden Funktionalitäten eines E-Commerce-Systems abbilden, von der Produktverwaltung bis zum Bestellprozess. Der Fokus liegt dabei auf einer sauberen, erweiterbaren Softwarearchitektur und der klaren Definition von Schnittstellen, insbesondere zwischen Backend und Frontend. Das Projekt soll eine vertiefte Auseinandersetzung mit Entwurfsmustern und den Prinzipien von Softwarearchitekturen fördern, die in der Vorlesung besprochen wurden.

Überlegen Sie sich anhand dieser Projektbeschreibung, welche Klassen und Attribute Sie in Ihrem Datenmodell abbilden müssen, um die Funktionalität der Anwendung zu erreichen.

2. Allgemeine Konzepte, Funktionalitäten und Rollen

2.1. Allgemeine Konzepte

Die Anwendung verwaltet einen Produktkatalog, Benutzerkonten und einen einfachen Bestellprozess. Dabei gibt es vier Arten von **Benutzer:innen** (siehe Abschnitt 2.2 für weitere Informationen):

- Besucher:innen (nicht registrierte Benutzer:innen)
- Kund:innen (registrierte Benutzer:innen)
- Shop-Manager:innen
- Administrator:innen

Der Kernbestandteil eines jeden Online-Shops sind die **Produkte**. Produkte sollen sowohl von Shop-Managern als auch von Administratoren bearbeitet, hinzugefügt oder gelöscht werden können.

Produkte sollen mindestens die Attribute `name`, `preis`, `bestand`, `rabatt` (prozentual), sowie ein `produktBild`¹ enthalten.

Registrierte Benutzer:innen sollen die Möglichkeit haben, ein Produkt (bzw. eine beliebige Stückzahl) in einen persönlichen **Warenkorb** zu legen. Der Warenkorb ist die Vorstufe zur eigentlichen Bestellung und umfasst ein oder mehrere Produkte samt deren Anzahl. Ein unbestellter Warenkorb soll nach Schließen und erneutem Öffnen der Webanwendung weiterhin vorhanden sein. Überlegen Sie sich, ob Sie den Warenkorb als eigenes Objekt in der Datenbank oder als Liste von Produkten im React Frontend (z.B. im `local storage`) implementieren. Wählen und begründen Sie Ihre Entscheidung in der Dokumentation. Besucher:innen des Online-Shops haben die Möglichkeit, Produkte nach verschiedenen Kriterien zu filtern (z.B. nach Bewertung, Preis, Stückzahl, etc.). Auch das Anwenden mehrerer Filter auf einmal soll möglich sein.

Wenn ein:e Benutzer:in eine **Bestellung** aufgibt, wird der Inhalt des Warenkorbs in eine feste Bestellung umgewandelt. Bestellungen enthalten eine `bestellNummer`, den `bestellStatus`, einen `zeitstempel` und die `summe`. Das System versendet bei Bestellung automatisch eine Bestätigungs-E-Mail mit den Bestelldetails inklusive Rechnung an die/den Benutzer:in und der Lagerbestand der bestellten Produkte wird entsprechend aktualisiert. Der Lagerbestand ist beim Abschluss einer Bestellung konsistent zu reservieren bzw. abzubuchen. Wenn ein Produkt nicht mehr verfügbar ist, soll es auch nicht mehr bestellt werden können.

Achtung

Es soll keine echte Zahlungsabwicklung implementiert werden. Die Benutzer:innen erhalten lediglich eine Rechnung per E-Mail als *plain-text*. Für dieses Projekt ist es weiter nicht verpflichtend, E-Mails zu versenden. Sie können diese Funktionalität als Stub² implementieren (z.B. durch Aufruf einer `stubbed sendMail()` Methode). Eine produktionsreife SMTP-Konfiguration ist also nicht erforderlich, die Architektur und Logik des E-Mail-Versands sollten jedoch korrekt implementiert sein.

Registrierte Benutzer:innen können ein Produkt bewerten³. Eine **Bewertung** soll Benutzer:innen ermöglichen, anhand einer Skala (z. B. 1–5) und eines Kommentars ihre Meinung zum Produkt zu äußern. Auch ein `zeitstempel` soll bei der Erstellung einer Bewertung gespeichert werden.

Ein zentrales Feature der Anwendung ist das **Benachrichtigungssystem**. Benutzer:innen sollen die Möglichkeit haben, sich bei bestimmten Produkt-Ereignissen (z.B. wenn ein ausverkauftes Produkt wieder verfügbar ist oder wenn ein Produkt im Preis gesenkt wird) informieren zu lassen. Während Sie in diesem Projekt lediglich Benachrichtigungen mittels E-Mails implementieren müssen, soll das System das Versenden von Benachrichtigungen über verschiedene Kanäle erlauben. Das heißt, es sollen weitere Kanäle wie z.B. WhatsApp oder SMS hinzugefügt werden können, ohne den bestehenden Code grundlegend ändern zu müssen. Erstellen Sie Stubs für eine SMS-Benachrichtigung, um zu zeigen, wie Sie die weiteren Benachrichtigungsmethoden implementieren würden.

¹Für dieses Projekt ist es nicht verpflichtend, ein Produktbild zu speichern. Sie können diese Funktionalität als Stub implementieren.

²Ein Stub bezeichnet einen üblicherweise relativ einfachen und kurzen Programmcode, der anstelle eines anderen, meist komplexeren, Programmcodes steht [Wikipedia].

³Das Überprüfen, ob die Benutzer:in das Produkt bereits gekauft hat, ist optional und muss nicht implementiert werden.

2.2. Rollen und deren Funktionalitäten

Die Anwendung unterteilt sich in vier Rollen mit unterschiedlichen Berechtigungen:

- **Administrator:innen** sind für die grundlegende Systemverwaltung zuständig.
 - Erstellen und verwalten Shop-Manager:innen-Konten.
 - Besitzen alle Rechte von Shop-Manager:innen.
- **Shop-Manager:innen** sind für die Pflege des Produktkatalogs verantwortlich.
 - Verwalten den Produktkatalog (Produkte anlegen, bearbeiten, löschen).
 - Passen Produktattribute wie Preis, Beschreibung und Lagerbestand an.
- **Kund:innen** des Webshops (registrierte Benutzer:innen).
 - Fügen Produkte in den persönlichen Warenkorb hinzu und verwalten diesen.
 - Bearbeiten Ihrer persönlichen Daten (Vorname, Nachname, E-Mail-Adresse, Versandadresse, etc.).
 - Schließen einen Bestellvorgang ab und lösen damit den Versand der Bestätigungs-E-Mail aus.
 - Bewerten Produkte.
 - Abonnieren Benachrichtigungen für einzelne Produkte, um über Änderungen informiert zu werden.
- **Besucher:innen** können den Shop erkunden, müssen sich jedoch registrieren, um Produkte zu kaufen.
 - Durchsuchen und betrachten den gesamten Produktkatalog.
 - Registrieren sich, um ein Benutzerkonto zu erstellen und weitere Funktionalitäten freizuschalten.

3. Architektur und Komponentenstruktur

Zentraler Fokus dieses Projekts ist eine saubere Architektur mit klaren Verantwortlichkeiten, hoher Kohäsion und geringer Kopplung. Strukturieren Sie Ihr System entlang der 3-Schichten-Architektur (Presentation/Frontend, Application/Service, Persistence).

3.1. Schnittstellen zwischen Backend und Frontend

Definieren Sie explizit alle benötigten REST-Endpunkte inkl. Request/Response-Strukturen. Geben Sie dabei Pfade, HTTP-Methoden, Parameter, Statuscodes und Beispiel-Payloads an. Erstellen Sie eine eigenständige Schnittstellenspezifikation und legen Sie diese im GIT-Repository bis [14.12.2025](#) ab (z.B. unter docs/). Ein Template wird im OLAT bereitgestellt. Diese Spezifikation wird am 15. Dezember im Proseminar besprochen und dient als Grundlage für den Datenaustausch zwischen Front- und Backend. Halten Sie Ihre Schnittstellen im weiteren Verlauf ein und besprechen Sie jede Änderung mit Ihren Teammitgliedern.

3.2. Entwurfsmuster

Wenden Sie geeignete Design-Patterns gezielt an und dokumentieren Sie kurz, wo und wie diese umgesetzt sind. Sehen Sie sich zur Hilfe noch einmal die **VO-Folien 04** an. Eine weitere empfehlenswerte Ressource ist die Seite [refactoring.guru](#). Sie müssen zumindest das **Observer** Design-Pattern im Benachrichtigungssystem bewusst umsetzen und in der Dokumentation beschreiben. Natürlich können Sie auch für andere Teile des Systems geeignete Design-Patterns umsetzen (wie z.B. **Decorator**).

4. Weitere Anforderungen

4.1. Allgemeine Anforderungen

Für die Abgabe des Projekts müssen alle beschriebenen Funktionalitäten implementiert werden und das System lauffähig sein. Zusätzlich werden an das System folgende allgemeine Anforderungen gestellt:

- Das System *muss* mit den aktuellen Versionen von modernen Browsern (Chromium, Firefox) kompatibel sein.
- Alle Listenansichten *müssen* filter- und sortierbar sein.
- Das System *muss* auf unerwartete Systemzustände (Exceptions) angemessen reagieren und den Benutzer:innen entsprechende **Fehlermeldungen zurückliefern**. Eine generische Fehlermeldung, wie z.B. „ein unerwarteter Fehler ist aufgetreten“, ist für erwartbare Ereignisse unzureichend.
- Das System *soll* benutzerfreundlich (Usability) gestaltet und intuitiv bedienbar sein.
- Das System *muss* **Multi-User** fähig sein: Jede:r Benutzer:in des Systems *muss* mit Username, Passwort (encoded), Vorname, Nachname, Rolle und E-Mail-Adresse gespeichert werden. Benutzer:innen müssen sich immer erfolgreich anmelden, bevor sie Produkte kaufen können.
- Das System *muss* das Löschen von Entitäten (Benutzer, Produkte, ...) unterstützen. Dabei ist zu beachten, dass das Löschen von bestimmten Entitäten weitere Auswirkungen auf das System haben kann (z.B. wenn ein:e Kund:in gelöscht wird, was passiert mit deren Warenkorb bzw. deren Bestellungen?). Hierfür *müssen* geeignete „**Lösch-Policies**“ definiert, dokumentiert und umgesetzt werden. Beispiele hierfür sind ein **soft-delete**, wobei die Entität in der Datenbank nicht gelöscht wird, sondern ein Flag gesetzt wird, um zu verhindern, dass sie bei der nächsten Abfrage wieder gelesen wird. Oder ein **hard-delete**, wobei die Entität in der Datenbank vollständig gelöscht wird.

4.2. Technische Anforderungen

Die Anwendung muss in einer 3-Schichten-Architektur entwickelt werden. Verwenden Sie das Spring Framework, insbesondere Spring Core, Spring Data und JPA.



Hinweis

In der Vorlesung wurde Ihnen ein Demo-Skeleton-Projekt⁴ vorgestellt, welches als Frontend React verwendet. Die Verwendung des Spring-Backends ist verpflichtend und die Verwendung des mitgelieferten Frontend-Codes ist stark empfohlen. Grundsätzlich steht es Ihnen aber frei, ein anderes UI-Framework zu wählen. Sie müssen allerdings in der Lage sein, dieses selbstständig mit Spring zu konfigurieren. Bei etwaigen Fragen werden wir uns zwar bemühen, Ihnen zu helfen, können aber keine Unterstützung garantieren.

Das umfangreiche und automatisierte Testen einer Applikation ist essenziell. **Testen Sie Ihren Source-Code** mit realitätsnahen Testdaten in ausreichender Qualität und Quantität durch Unit- und Integration-Tests. Besonders für die zentrale Geschäftslogik ist eine gute Testabdeckung wichtig. Verwenden Sie SonarQube, um die Qualität Ihres Quellcodes regelmäßig zu überprüfen. Achten Sie außerdem darauf, die gefundenen Issues (Bugs, Vulnerabilities, Code-Smells, etc.) zeitnah zu beheben. Eine SonarQube-Instanz samt Anleitung, wie Sie diese in Ihrem GIT-Repository einrichten

⁴https://git.uibk.ac.at/informatik/qe/swa_swe/swa/swa-skeleton.

und verwenden, ist im GIT-Repository des Skeleton-Projekts enthalten. Bitte melden Sie sich bei Fragen zur Einrichtung und Verwendung von SonarQube bei der Lehrveranstaltungsleitung.

Automatisierte UI- und API-Tests sind nicht erforderlich. Das bedeutet, dass Sie keine Tests für Frontend-Code schreiben müssen. Ebenfalls müssen Sie die API nicht mit Postman o. Ä. testen. Es macht dennoch Sinn, das Frontend durch SonarQube untersuchen zu lassen, um über Code-Smells informiert zu bleiben.

5. Organisatorisches und Entwicklungsrichtlinien

Die Projektarbeit erfolgt im Team (4-5 Personen). Wir ersuchen Sie, im Proseminar am 1.12.2025 Ihr Team und jeweils eine Ansprechperson Ihrer Lehrveranstaltungsleitung per E-Mail mitzuteilen. Die LV-Leitung wird Ihnen dann ein GIT-Repository zur Verfügung stellen, in dem Sie alle Ihre erstellten Artefakte (z. B. Quellcode Dateien, Diagramme, etc.) ablegen müssen. Beachten Sie dabei unter anderem folgende Punkte:

- Verwenden Sie das im Laufe des Semesters vorgestellte **GIT Branching Modell**⁵ mit main, dev und Feature-Branches.
- Halten Sie sich bei der Programmierung an die in der Vorlesung besprochenen Java und React Code Conventions.
- Benennen Sie Commit-Messages sinnvoll⁶.
- Achten Sie auf eine **sinnvolle und aussagekräftige Dokumentation Ihres Sourcecodes** (Eine automatische Generierung von JavaDoc Kommentaren ist nicht zielführend!). Grundsatz: Ein:e neue:r Mitarbeiter:in sollte sich anhand der Dokumentation in das Projekt einarbeiten können. Die Dokumentation muss zudem eine **Beschreibung, wie man das Projekt aufsetzt und ausführt** beinhalten.

5.1. Weitere wichtige Hinweise

Die Teamarbeit steht bei diesem Projekt im Vordergrund, die Lehrveranstaltungsleiter achten besonders auf die gleichmäßige Verteilung von Aufgaben im Team. **Konflikte innerhalb des Teams sollten frühestmöglich mit der Proseminarleiter:in besprochen werden. Die Arbeitsleistung der einzelnen Teammitglieder wird AUSSCHLIESSLICH durch direkte Beiträge (Commits, Code Contributions) festgestellt.** Pair-Programming, bei dem nur eine Person programmiert, wird ausschließlich für die Person gewertet, die den Commit erstellt. Beachten Sie, dass sich die Commit-Autor:in beim Überschreiben von Commits (durch rebase / squash, aber auch merge / squash) ändern kann. Auch in diesem Fall wird der Commit der Autor:in angerechnet, die zum jeweiligen Abgabepunkt Hauptautor:in ist.

Im Normalfall werden Punkte einheitlich für das gesamte Team vergeben. Stellt sich jedoch heraus, dass sich einzelne Studierende besonders viel oder wenig im Team einbringen, kann auch eine vom restlichen Team unabhängige Punkteanzahl vergeben werden.

Alle in diesem Dokument aufgeführten und direkt daraus ableitbaren Arbeitsabläufe, Features und Anforderungen sind zwingend, vollständig und in angemessener Qualität für eine positive Beurteilung des Projekts umzusetzen! Die LV-Leitung räumt sich das Recht ein, dass die von Studierenden erstellten Artefakte in anonymisierter Form für Lehre und Forschung verwendet werden können. Insbesondere können Projektergebnisse (und Teile davon) für Ausbildungszwecke eingesetzt werden.

⁵Slidedeck 05 im [OLAT](#).

⁶<https://xkcd.com/1296/>.

! Achtung

Sollten Sie Fragen zur Aufgabenstellung oder dem Technologieeinsatz haben, stellen Sie diese bitte im OLAT Forum.

5.2. Umgang mit Large Language Models (LLMs)

Der Einsatz von LLMs ist erlaubt, sofern er reflektiert und verantwortungsvoll erfolgt. Achten Sie dabei auf:

- Einen zielgerichteten Einsatz (z. B. Code-Reviews, Refactoring-Vorschläge, Testfallideen).
- Transparenz: Dokumentieren Sie relevante Prompts und Ergebnis-Auszüge in einem kurzen Erfahrungsbericht (1–2 Seiten).
- Rechtliche Aspekte: Achten Sie auf Urheberrecht, Lizenzen, Datenschutz und Vermeidung von sensiblen Daten in Prompts.
- Kritische Prüfung: Übernehmen Sie generierte Artefakte nur nach Review, Tests und Qualitätsprüfung. **Sie müssen generierte Artefakte verstehen und beschreiben können!**

5.3. Erwartete Deliverables

Für dieses Projekt sind folgende Artefakte verpflichtend im GIT-Repository abzulegen (z.B. unter docs/):

Frist	Artefakt	Beschreibung
14.12.2025 23:59	Schnittstellenspezifikation	Backend - Frontend Spezifikation mit Endpunkten, Parametern, Schemata, Statuscodes und Beispielen. Ein Template wird im OLAT bereitgestellt.
	Fachliches UML-Klassendiagramm	Fachliches UML-Klassendiagramm des in diesem Dokument beschriebenen Systems.
	UML-Sequenzdiagramm	UML-Sequenzdiagramm, welches den Workflow für folgende Situation abbildet: Das Produkt im Warenkorb ist beim Kauf nicht mehr verfügbar.
	Issues und Meilensteine	In GitLab erstellte Issues und Meilensteine.
02.02.2026 08:00	Source Code	Quellcode des Systems.
	Dokumentation	Dokumentation des Systems inklusive der Lösch-Policies und der Beschreibung, wie man das Projekt aufsetzt und ausführt. Zusätzlich soll dokumentiert werden, welche Design-Patterns wo und wie umgesetzt wurden.
	GenAI-Nachweis	Kurzbericht (1–2 Seiten) inkl. relevanter Prompts und Resultatauszüge.

5.4. Fristen, wichtige Termine und Abgaben

Frist	Beschreibung
Am 01.12.2025 im PS	Besprechung des Projekts, dessen Anforderungen und Bewertungskriterien.
Bis 04.12.2025 12:00	Bilden Sie Teams (4-5 Personen) und legen Sie eine Ansprechperson fest. Senden Sie eine E-Mail an Ihre Lehrveranstaltungsleitung mit den Teammitgliedern und der Ansprechperson.
Bis 14.12.2025 23:59 (Abgabe im GIT)	Erstellen Sie einen Projektplan bestehend aus • einem fachlichen UML-Klassendiagramm und einem Sequenzdiagramm (als PDF in GIT ablegen). Das Sequenzdiagramm soll den Workflow für die <i>Bestellung eines Produkts im Warenkorb, das nicht mehr im Lager verfügbar ist (Lagerbestand in der Datenbank == 0.)</i> abbilden. • Abgabe der Schnittstellenspezifikation (Expected Deliverable) im GIT-Repository. • Meilensteine, Issues und deren Verantwortlichkeiten (In GitLab) Folgende Meilensteine müssen verpflichtend im Projektplan berücksichtigt werden: • Meilenstein 1: 12.01.2026 • Meilenstein 2: 30.01.2026 Der Inhalt der Meilensteine bleibt Ihnen überlassen. Gerne können Sie noch zusätzliche Meilensteine anlegen.
Am 15.12.2025	Architektur-Check: Diskussion der vorgesehenen Architektur mit Fokus auf Komponentenstruktur, Kohäsion/Kopplung und Entwurfsmuster.
Am 12.01.2026, 19.01.2026, und 26.01.2026, jeweils 08:00	Es wird ein kontinuierlicher Projektfortschritt verlangt. Der Projektfortschritt wird anhand des GIT-Projekts (z. B. Inhalt, Projektplan) durch die LV-Leiter:innen vor jedem Termin bewertet. Es zählt nur, was bis spätestens 08:00 Uhr am Tag der jeweiligen LV-Einheit in den main-Branch gemerged ist. Bei zweimaliger Verletzung dieser Anforderung erfolgt eine negative Beurteilung des Projektteams bzw. der Studierenden.
Am 02.02.2026 08:00	Der Stand um 8:00 im main-Branch wird für die Bewertung der Projektarbeit herangezogen. Bitte beachten Sie, dass Ihre Applikation mit der im Skeleton-Projekt definierten Maven-Konfiguration und den Befehlen <code>mvn spring-boot:run</code> sowie <code>npm start</code> ausführbar sein muss. (vergleiche Sektion Anforderungen) Bitte vergessen Sie nicht, das fachliche UML-Klassendiagramm, das fachliche Sequenzdiagramm und den Projektplan ebenfalls in Ihrem GIT Repository abzulegen.
Bis 02.02.2026 im PS	Fertigen Sie eine Präsentation (max. 10 Minuten) mit interessanten Details zu Ihrem System an, die Ihr Team im Proseminar präsentieren wird. Weitere Details zur finalen Präsentation und zum genauen Ablauf werden noch bekannt gegeben.

6. Hilfreiche Referenzen und Tools

Die folgenden Referenzen könnten bei der Bearbeitung der Projektarbeit hilfreich sein, sind aber nicht verpflichtend:

Allgemein

- VO Folien
- Kollaborationswerkzeuge (Jitsi, Slack, Discord, Tisch)
- Maven Dokumentation ([Link](#))

- Beispielhafte arc42 Dokumentation ([Link](#))
- Java ist auch eine Insel, C. Ullenboom ([Link](#))

GIT

- GIT Dokumentation ([Link](#))
- GIT Cheatsheet ([Link](#))

Spring Framework

- Dependency Injection ([Link](#))
- Bean Scopes ([Link](#))
- Deklaration von Beans ([Link](#))
- Spring Data - Zugriff auf Daten ([Link](#))
- Spring Data – Zusammenspiel Repository und Service ([Link](#))
- Spring Web ([Link](#))
- Spring REST Tutorial ([Link](#))
- Baeldung REST Serie ([Link](#))

React

- React-Dokumentation (<https://react.dev/>)
- React TypeScript Cheatsheet ([Link](#))
- LearnXinYMinutes ([Link](#))
- PrimeReact (<https://primereact.org/>)

API Tools

- Bruno (<https://www.usebruno.com>)
- Postman (<https://www.postman.com/>)

Dokumentation mittels JavaDoc

- Dokumentationskommentare mit JavaDoc ([Link](#))

UML-Modellierung

- UML-Spezifikation ([Link](#))
- Hilfreiche UML-Webseite ([Link](#))

Beispiele für UML-Tools

- Modelio ([Link](#))
- Lucidchart ([Link](#))
- Eclipse Papyrus ([Link](#))
- UML Let ([Link](#))
- Diagrams.net (<https://app.diagrams.net/>)

Dokumentation von Softwarearchitekturen

- Offizielle arc42 Webseite ([Link](#))